SOFTWARE REQUIREMENTS & DESIGN SPECIFICATION · V1.0

TriageZero System Specification

An autonomous platform that investigates regression-test failures: it receives structured evidence from a test suite, determines whether the application or the test is at fault, assesses release risk, and recommends an action for a human to approve.

Status Deployed & operating autonomously **Date** 30 August 2026 **Platform** Google Cloud

Repository bradhak5-ASU/TriageZero-AI

CONTENTS

1 Problem statement

An automated regression suite fails overnight. An engineer arrives to a red build and spends the first hour of the day answering one question before any real work begins.

That question is: **is this a defect in the application, or did the test itself break?** Answering it means opening a trace, correlating network responses against console output, checking whether a selector still matches the DOM, comparing against yesterday's run, and deciding whether the failure should block a release.

The work is repetitive, requires no creativity, and consumes senior engineering attention at the worst possible moment — the start of the day, on a blocked pipeline. It also scales badly: a suite that fails in five places produces five instances of the same investigation.

TriageZero performs that first hour automatically. It receives the evidence a failing test already produces, reaches a diagnosis with stated confidence and reasoning, assesses release risk, retrieves comparable historical failures, and proposes an action. A human approves or rejects; the system never acts unilaterally.

> SCOPE BOUNDARY
>
> TriageZero does not run tests, fix code, or merge anything. It consumes failure evidence and produces a reviewed recommendation. The test suite remains the customer's; the judgement remains the engineer's.

2 Stakeholders

Four groups interact with the system, with materially different needs and access.

STAKEHOLDER	WHAT THEY NEED	HOW THEY ACCESS IT
QA / test engineer	To know within seconds whether a red build is their test or the developers' code, with evidence they can check.	Dashboard, signed in with Firebase Authentication.
Developer on call	A specific implicated component and a next step, not a stack trace to re-read.	Dashboard investigation detail.
Release manager	Whether anything currently failing should block the release, and who signed off.	Release-risk view; approval history is recorded per action.
The CI system `machine`	To submit a failure package and get an identifier back, unattended.	Ingestion API with a dedicated machine token — never a human credential.

The distinction between the last row and the rest is enforced, not conventional: machine and human credentials are verified independently and cannot substitute for one another (see §10).

3 How it works

One failing test, end to end, with no human in the loop until the final approval.

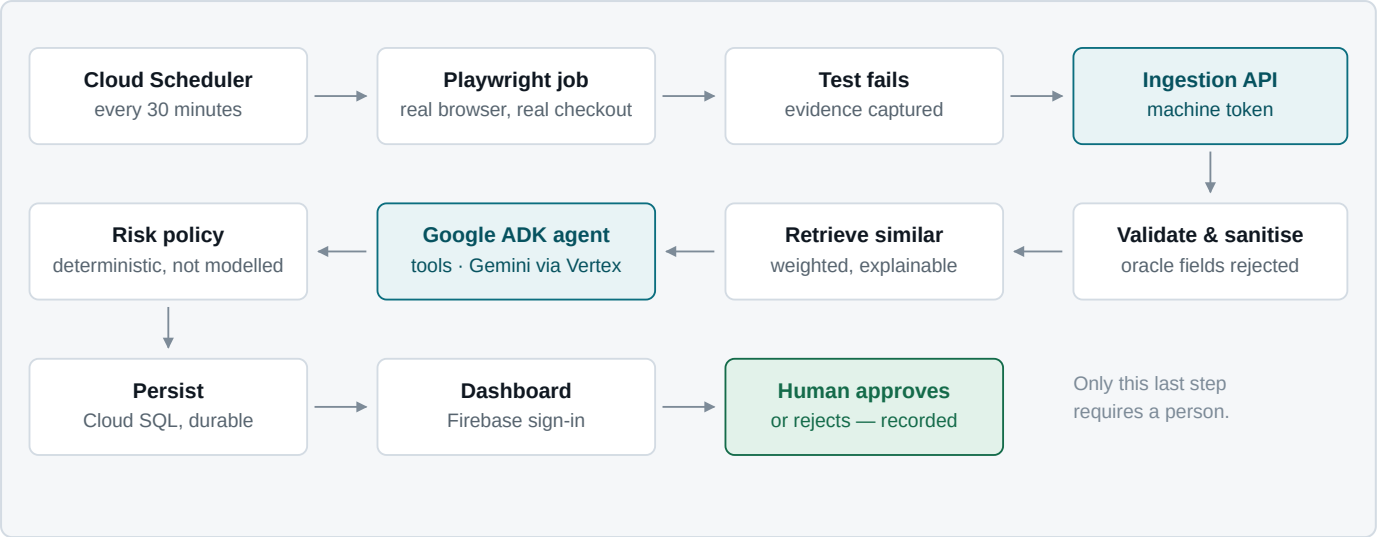

Figure 1 — The autonomous loop. Every step above the final approval runs unattended on a 30-minute schedule.

The evidence a failing test carries is specific: failing network requests with status codes, browser console output, the assertion's expected and actual values, a stack trace, a screenshot and a Playwright trace. The analyzer reasons over that evidence and nothing else — it has no access to the codebase, the filesystem, or the network.

4 Functional requirements

Status reflects what is running in the deployed system on the date of this document, not what the code supports.

ID	REQUIREMENT	STATUS
FR-1	Accept a structured failure package over HTTP from an automated test suite, returning an investigation identifier.	Verified in production
FR-2	Reject any package containing private QA-oracle fields, at any nesting depth, before parsing.	Verified in production
FR-3	Classify each failure into a closed vocabulary of eight categories, including distinguishing an application defect from a test-automation defect.	Verified in production
FR-4	Produce a root-cause hypothesis, the implicated component, a confidence score and an explanation of that confidence.	Verified in production
FR-5	Assess release risk on a five-point scale, computed deterministically rather than taken from the model.	Verified in production
FR-6	Retrieve comparable historical failures with the matching signals that justify each match.	Verified in production
FR-7	Recommend an action and require explicit human approval before it is considered taken.	Verified in production
FR-8	Deduplicate identical failure packages and reject a reused idempotency key carrying different evidence.	Verified in production
FR-9	Run the test suite on a schedule and ingest its failures with no human present.	Verified in production
FR-10	Record a human's review outcome as the only source of ground truth for evaluation.	Implemented, unexercised
FR-11	Create issues in an external tracker from an approved action.	Not implemented

FR-10 is implemented and reachable in the dashboard, but no human review has been recorded yet, so the historical corpus is empty. FR-11 was scoped out deliberately: the system records decisions rather than acting on external systems.

5 Non-functional requirements

ID	REQUIREMENT	HOW IT IS MET	STATUS
NFR-1	Investigations survive restart, redeploy and scale-to-zero.	Cloud SQL PostgreSQL. The service refuses to start on SQLite in production.	Verified
NFR-2	The dashboard is unreadable without authentication.	Firebase ID token or dashboard service token; 401 otherwise.	Verified
NFR-3	A machine credential cannot read human data.	Scope separation returns 403, not 401, on the wrong route.	Verified
NFR-4	The API is callable only from the dashboard's exact origin.	Explicit CORS origin; wildcards and plaintext rejected at startup.	Verified
NFR-5	A model provider failure degrades the system rather than breaking it.	Deterministic rule engine as fallback, labelled visibly in provider metadata.	Verified
NFR-6	Evidence cannot instruct the analyzer.	Delimited untrusted evidence, no tool access to external systems, schema-validated output.	Verified
NFR-7	No secret appears in the repository, an image layer, or the browser bundle.	Secret Manager injected at container start; scanned across all commits.	Verified
NFR-8	An analysis cannot hang indefinitely.	Per-analysis deadline plus a 40-event budget on the agent loop.	Verified
NFR-9	Health reporting must describe the system as deployed, not as developed.	Datastore, region and provider state derived from live configuration.	Verified
NFR-10	Request bodies are size-limited even without a declared length.	Content-Length gate plus a received-bytes cap for chunked requests.	Verified

6 Architecture

Six Cloud Run workloads, one PostgreSQL instance with two isolated databases, and two schedules — all in a single Google Cloud project.

WORKLOAD	TYPE	ROLE	SCALING
triagezero-web	Service	Dashboard — static bundle behind nginx	min 1
triagezero-api	Service	Ingestion, analysis, persistence	min 1
novacart-web	Service	Demo shopfront — the application under test	min 0
novacart-api	Service	Demo shop backend, with injectable defects	min 0
triagezero-scheduled-tests	Job	Playwright suite against the deployed shop	on schedule
novacart-seed	Job	Restores demo catalogue and stock	on schedule

Separation of platform and subject

The application under test is deliberately isolated from the platform that investigates it. NovaCart runs under its own service account, connects as its own database user, and holds one credential to one database. It cannot reach the investigation store or any TriageZero secret — which matters, because the thing being tested is, architecturally, untrusted input.

> **WHY ONE DATABASE INSTANCE, TWO DATABASES**
>
> A second Cloud SQL instance would double the largest cost line for no isolation benefit that separate databases and separate users do not already provide. The boundary is enforced by credentials, not by hardware.

Why the analyzer is an abstraction

Three interchangeable implementations satisfy one interface and return one validated result: a deterministic rule engine, a direct Gemini call, and a Google ADK multi-stage agent. The active one is chosen by configuration. This is what makes the fallback in NFR-5 possible, and it is what allowed the ADK path to be developed and debugged without the deployed system ever losing the ability to analyse a failure.

7 Technology stack

LAYER	CHOICE	WHY
Dashboard	React 18, Vite 8, TypeScript 5.6, React Router 7	Static bundle, no server runtime to secure or scale.
Dashboard tests	Vitest 4, Testing Library	Behavioural tests against rendered output rather than implementation.
API	FastAPI, Python 3.11, Uvicorn	Typed request models are the validation layer, not an addition to it.
Validation	Pydantic v2, closed schemas	`extra="forbid"` makes an unexpected field an error rather than data.
Persistence	SQLAlchemy 2, PostgreSQL 16 / SQLite	One model layer across the cloud store and local development.
AI	Google Gen AI SDK 1.75, Google ADK 1.35	Agent framework plus direct model access behind one interface.
Authentication	Firebase Authentication, Firebase Admin	Human identity handled by a service built for it; tokens verified locally.
Test runner	Playwright 1.62	Real browser, real network capture — the evidence is the product's input.
Quality gates	pytest, Ruff, ESLint 9, tsc	All four run in the build before anything is deployed.

8 Google products

PRODUCT	HOW IT IS USED
Google ADK	The analysis agent. Five read-only evidence tools — network, console, failure text, similar-case retrieval, risk calculation — with a stopping rule and an event budget. This is the component that performs the investigation.
Vertex AI	Serves Gemini to the agent using the service account's own credentials. No API key exists anywhere in the system.
Cloud Run	Four services and two jobs. Services scale to zero where appropriate; jobs are the unit of scheduled work.
Cloud SQL	PostgreSQL 16. The durable store — the reason investigations survive a redeploy.
Cloud Scheduler	Two schedules: the test run, and a restock five minutes ahead of it.
Cloud Build	Runs both test suites, then builds, then deploys. A failing test stops the deploy.
Secret Manager	Four secrets, each readable only by the service accounts that need it.
Firebase Authentication	Email/password sign-in for humans, verified server-side against this project.
Artifact Registry	Container images for every workload.
Cloud Logging	Structured JSON logs. Every diagnosis in this project's development came from these.

9 Data contract

The failure package is a versioned, closed schema. Any field the schema does not name is a validation error, which is what allows the system to accept input from a test suite it does not control.

Analysis output

The analyzer must return exactly fourteen fields — classification, confidence, severity, release risk, root-cause summary, implicated component, confidence explanation, evidence highlights, next step, recommended action, action rationale, proposed issue title, proposed labels, and a human-review flag. Anything resembling free-form reasoning is rejected.

Closed vocabularies

FIELD	PERMITTED VALUES
Classification	`backend_application_defect · frontend_application_defect · test_automation_defect · environment_failure · data_integrity_defect · performance_timing_defect · dependency_failure · unknown`
Severity	`critical · high · medium · low`
Release risk	`block_release · high · moderate · low · none`
Provider	`deterministic · gemini · gemini_adk · deterministic_fallback`

The provider field is recorded on every investigation. A reader can always tell which analyzer produced a given conclusion, including when a fallback occurred.

10 Security model

Two credential types that cannot substitute

Humans present a short-lived Firebase ID token. Machines present a long-lived ingestion token. These are verified by different code paths, and presenting one on the other's route returns **403, not 401** — the distinction matters, because 401 invites a retry with a better credential, while the correct answer is that this credential is not for this route.

Least privilege, per workload

SERVICE ACCOUNT	PERMISSIONS
`triagezero-api`	Cloud SQL client, Vertex AI user, read access to its own three secrets — not to secrets in general.
`triagezero-web`	None. It serves static files and holds no credential.
`novacart-app`	Cloud SQL client and one secret, for its own database only.
`triagezero-runner`	Read one secret — the ingestion token. Nothing else.
`triagezero-scheduler`	Invoke two named jobs. No data access at all.

Prompt-injection posture

Failure evidence is attacker-influencable — a test can be made to log anything. Four controls apply: evidence is delimited and labelled untrusted; the agent has no tools that reach outside the evidence; output is validated against a closed schema before persistence; and release risk is computed deterministically rather than taken from the model, so no text in the evidence can talk the system into declaring a critical failure safe.

> ON THE DEMO SHOP BEING PUBLIC
>
> The application under test is intentionally open — it holds fabricated products in its own database and must be reachable by the test runner. Its exposure grants no access to the platform.

11 Verification

BACKEND TESTS	FRONTEND TESTS	BACKEND MODULES	TEST SUITES	COMMITS
284	**60**	**51**	**16**	**25**

The backend suite runs against a real PostgreSQL server, not a mock: legacy-schema migration with duplicate keys repaired and no rows lost, the unique index actually rejecting a duplicate, migration idempotency, and the full ingestion flow including the

idempotency conflict.

Verified in the deployed system

- Container starts on the injected port; readiness reports the datastore reachable and migrated.
- Unauthenticated dashboard request returns 401; an ingestion token on a dashboard route returns 403.
- CORS echoes the exact dashboard origin and refuses any other.
- A real failure package ingested, analysed and persisted — then the process killed and restarted, with the investigation intact.
- A scheduled run, triggered by the scheduler's own service account rather than a person, producing correctly classified investigations.

Accuracy on real failures

The synthetic holdout is a regression tripwire, not an accuracy claim: its generator builds scenarios out of the same signal vocabulary the deterministic rules encode, so scoring 1.00 there is close to tautological. Accuracy was therefore measured a second way — on failures the platform produced itself.

`scripts/measure_field_accuracy.py` reads every non-synthetic investigation in the deployed database and labels each one from evidence that exists independently of the analyzer: **the browser recorded an HTTP 5xx from the application**. A 5xx is the server admitting it failed. Playwright captured it, it travels in the failure package, and it is true whether or not TriageZero exists — no rule, prompt or heuristic in this repository has any say in it. The label is fixed before the analysis runs, which is what makes it ground truth rather than a restatement of the model's own reasoning.

ACCURACY	LABELLED CASES	ADK AGENT	DISAGREEMENTS	MEAN CONFIDENCE
100%	**84** / 84 correct	**80** / 80	**0**	**0.941**

ANALYSIS PROVIDER	CORRECT	CASES	ACCURACY
`gemini_adk` — Google ADK agent on Vertex AI	80	80	`100%`
`gemini` — direct Gen AI SDK	4	4	`100%`
Overall	**84**	**84**	`100%`

112 investigations were read from unattended scheduled runs against the deployed shop. 84 carried an externally decidable label; 28 did not, and were excluded rather than guessed.

The exclusions matter as much as the score. The undecidable 28 are the catalogue-exhaustion runs: the test fails at a disabled control with no failing request, and both “frontend defect” and “data defect” are defensible readings of that evidence. Scoring

them either way would be the experimenter choosing the answer, so they sit outside the denominator.

> WHAT THIS DOES AND DOES NOT CLAIM
>
> One defect class, 84 cases. It does not establish accuracy across the full eight-way classification space, and a second injected defect family would strengthen it. What it does establish is that on real browser evidence from a real deployment, with the answer fixed in advance, the agent was correct in every labelled case — and was calibrated rather than merely assertive.

> THE SYNTHETIC BENCHMARK STILL EARNED ITS KEEP
>
> Its first run scored macro-F1 0.6977 and failed its gate, because a benign console line was disqualifying a locator-timeout rule in 9 of 26 cases. **The fix went into the analyzer, not the dataset.**

12 Current status

The system is deployed and operating without human involvement. Every 30 minutes it tests a live application and files diagnosed investigations.

CAPABILITY	STATE
Dashboard, API and durable store deployed	`Operating`
Human sign-in, with machine credentials kept separate	`Operating`
Scheduled autonomous test runs	`Operating`
Google ADK agent performing the analysis	`Operating`
Classification accuracy measured on real failures	`84/84`
Deterministic fallback on provider failure	`Exercised`
Injectable defect scenarios for demonstration	`Six available`
Human review loop feeding the historical corpus	`Built, no entries`
Collector distributed as an installable package	`Not built`
External issue creation	`Out of scope`

An emergent finding worth recording

After several hours of unattended operation, investigations began arriving classified as frontend defects rather than the injected backend defect. The cause was not a misclassification: the scheduled runs had been placing real orders every 30 minutes and had exhausted the demo catalogue's stock, so the tests failed at the catalogue and never reached checkout. The agent diagnosed the symptom actually present in the evidence — a disabled control with no failing request — rather than the cause it was expected to find. A restock job now runs five minutes ahead of each test run.

13 Limitations

- **Field accuracy covers one defect class.** 84/84 on real failures with externally verifiable ground truth (§11), but from a single injected defect family; the full eight-way classification space is not yet covered.
- **The historical corpus is empty.** Similar-failure retrieval works, but has no human-reviewed cases to draw on yet, so its value grows only with use.
- **Onboarding another project is manual.** Two collector files must be copied into the target suite; there is no installable package.
- **The environment vocabulary is fixed** to local, staging and production. A project using other names would be rejected.
- **The collector is Playwright-shaped.** Other frameworks would need a mapping layer to produce the failure package.
- **Assertions cannot be generic.** A regression suite needs to know what the application should do; the platform is app-agnostic, the tests cannot be.

14 Future work

1. **Publish the collector** as an npm package or a Playwright reporter, reducing onboarding to an install and one configuration line.
2. **Inject a second and third defect family** — a slow dependency and a data-integrity fault — so the field measurement in §11 spans more of the classification space.
3. **Accumulate human-reviewed outcomes** so retrieval has genuine precedent and accuracy can be tracked against human verdicts as well as external evidence.
4. **Migrate schema changes to a Cloud Run job** executed before the service deploys, rather than relying on startup migration and the readiness gate.
5. **Adapters for other frameworks** — Cypress and Selenium — mapping their evidence into the same package.
6. **External tracker integration**, so an approved action creates the issue it proposes.

TriageZero — Software Requirements & Design Specification v1.0 · 30 August 2026.
Status reflects the deployed system on that date. Claims marked *verified* were confirmed by an executed command or test whose output was observed.